UNIDAD 2 · SESIÓN 7

JPQL y Transacciones con Spring Boot

Consultas seguras y operaciones confiables en el back-end

Desarrollo Web Integrado (100000ST61)
Ingeniería de Sistemas e Informática · Ingeniería de Software — Ciclo 6 y 7
Unidad 2: Back-end con bases de datos



INICIO

5 min · Trabajo en equipo

Rompe hielo: el reto del inventario fantasma



En equipos de 3 personas (2 minutos de discusión):

"Dos vendedores de TechStore Perú SAC, desde dos cajas distintas, venden el ÚLTIMO producto en stock casi al mismo instante. Ambos sistemas confirman la venta antes de que el inventario se actualice. ¿Qué creen que debería pasar para que TechStore no termine vendiendo un producto que ya no existe?"

- Discutan 2 minutos en su equipo.
- Un/a representante comparte la respuesta en voz alta (30 seg).
- No necesitan saber el término técnico todavía — solo la idea.

Pista para el docente

No corrijan aún. El objetivo es que los estudiantes intuyan el problema de concurrencia y consistencia de datos, sin usar todavía la palabra "transacción". Volveremos a esta idea en la fase de Transformación.

INICIO

3 min · Individual

Repaso rápido: Sesión 6 — JPA y Hibernate

Responde mentalmente antes de que revisemos juntos

1. ¿Qué anotación de JPA usamos para marcar una clase Java como una entidad de base de datos?

2. ¿Qué método del EntityManager usamos para guardar un nuevo objeto en la base de datos?

3. ¿Qué anotación de Spring registra una clase DAO para que Spring la administre automáticamente?

INICIO

4 min · Exposición docente

¿Por qué te debe importar esto?



El mismo mecanismo detrás de cada Yape

- Cada vez que pagas con Yape, Plin o una tarjeta, ocurre una operación de base de datos detrás de escena.
- Si la red falla a mitad de camino, el sistema NO puede dejar tu dinero “a medias”: descontado de un lado, pero no acreditado en el otro.
- Eso es exactamente lo que resolveremos hoy: cómo Spring Boot garantiza que las operaciones críticas se completen por entero, o no se apliquen en absoluto.
- Y JPQL será el lenguaje que usaremos para consultar y modificar esos datos de forma precisa y eficiente.



Hoy resolvemos:

- Cómo consultar datos relacionados de forma segura (JPQL).
- Cómo garantizar que operaciones críticas nunca queden “a medias” (@Transactional).

INICIO

3 min · Lluvia de ideas

Antes de comenzar, pensemos…

"Si tú fueras el/la desarrollador(a) de la app de Yape, y a mitad de una transferencia se corta la conexión a internet del celular… ¿qué esperarías que pase con el dinero del usuario? ¿Debería quedar descontado sin acreditarse en el otro lado?"

Recojamos 2 o 3 respuestas en voz alta.

Puente: hoy aprenderemos el mecanismo exacto que usa Spring Boot para evitar ese escenario.

UTILIDAD

5 min · Piensa y responde

Repaso retador: ¿lo tienes claro?

Preguntas de la Sesión 6 para pensar más allá de la definición

1. Si haces entityManager.persist(student) sobre una entidad con @OneToMany hacia Course, sin CascadeType.ALL configurado, ¿qué ocurre con los cursos asociados?

2. ¿Por qué NO es obligatorio @Transactional en un método que solo hace un SELECT con el EntityManager, pero SÍ lo es en uno que hace save(), merge() o remove()?

3. Si usas JpaRepository en lugar de EntityManager manual, ¿qué ventaja concreta obtienes al implementar operaciones CRUD básicas?

UTILIDAD

4 min · Revisión conjunta

Respuestas: repaso retador

- Los cursos NO se guardan automáticamente: sin cascada, esos objetos quedan “transient” (no gestionados) y no se persisten junto con el estudiante.
- El SELECT no modifica datos, por eso no requiere transacción; pero persist/merge/remove sí cambian el estado de la base de datos y deben ejecutarse dentro de una transacción para garantizar consistencia.
- JpaRepository ya trae implementados los métodos CRUD más comunes (save, findById, findAll, deleteById...), eliminando el código repetitivo que escribirías a mano con EntityManager.

UTILIDAD

3 min · Exposición docente

¿Qué lograrás al final de esta sesión?

Al finalizar la sesión, construirás consultas personalizadas con JPQL para recuperar y modificar información desde entidades JPA, y aplicarás el manejo de transacciones con @Transactional para garantizar que las operaciones críticas de negocio se ejecuten de forma completa, consistente y segura.

- Escribir consultas JPQL con SELECT, WHERE, parámetros con nombre y JOIN.
- Ejecutar actualizaciones y eliminaciones masivas de forma segura con @Modifying.
- Aplicar @Transactional para proteger operaciones críticas del negocio.
- Entender cuándo y por qué una transacción debe revertirse (rollback).

Esta es una habilidad indispensable en el mundo profesional: cualquier sistema que maneje pagos, inventarios o datos sensibles de usuarios depende de esto.

UTILIDAD

5 min · Exposición docente

¿Dónde se usa esto en el mundo real?



Yape / apps bancarias

Transferencias que nunca dejan el dinero “a medias” entre dos cuentas.



Rappi / e-commerce

Al confirmar un pedido: se descuenta stock, se crea el pedido y se registra el pago — todo o nada.



Spotify / Netflix

Consultas JPQL con JOIN traen canción + artista + álbum en una sola consulta eficiente.



Reservas y vuelos

Bloquean un asiento o una fecha de forma segura ante múltiples usuarios simultáneos.

En los próximos minutos aprenderás exactamente las herramientas que hacen esto posible.

TRANSFORMACIÓN

5 min

¿Qué es JPQL?

- JPQL (Java Persistence Query Language) es el lenguaje de consultas de JPA.
- Es similar en concepto a SQL: usa WHERE, LIKE, ORDER BY, JOIN, IN, etc.
- La diferencia clave: JPQL trabaja con entidades y sus atributos (clases Java), no con tablas y columnas de la base de datos.
- Esto hace que las consultas sean portables entre distintos motores de base de datos (MySQL, PostgreSQL, Oracle...).

SQL

```
SELECT * FROM student WHERE last_name = 'Doe'
```

tabla · columna

JPQL

```
SELECT s FROM Student s WHERE s.lastName = 'Doe'
```

entidad (clase) · atributo del objeto

TRANSFORMACIÓN

5 min

Sintaxis básica: SELECT · FROM · WHERE

```
TypedQuery<Producto> query = entityManager.createQuery(
    "SELECT p FROM Producto p WHERE p.categoria = 'Electrónica'",
    Producto.class);

List<Producto> productos = query.getResultList();
```

- "p" es un alias o identification variable: una referencia al objeto Producto dentro de la consulta.
- Para JPQL estricto, la cláusula SELECT es obligatoria (Hibernate, de forma flexible, también acepta omitirla).
- WHERE filtra igual que en SQL, combinando operadores AND, OR, LIKE, IN, BETWEEN.

TRANSFORMACIÓN

5 min

Parámetros con nombre (Named Parameters)

```
public List<Producto> buscarPorCategoria(String categoria) {
    TypedQuery<Producto> query = entityManager.createQuery(
        "SELECT p FROM Producto p WHERE p.categoria = :cat",
        Producto.class);
    query.setParameter("cat", categoria);
    return query.getResultList();
}
```

- El prefijo ":" define un parámetro con nombre — un espacio reservado que se llena luego.
- Evita concatenar cadenas de texto manualmente, lo que previene la inyección de código (SQL/JPQL injection).
- El valor real se asigna con setParameter("nombre", valor) antes de ejecutar la consulta.

TRANSFORMACIÓN

5 min

JOIN entre entidades relacionadas

Ejemplo: Producto pertenece a Categoria (@ManyToOne)

```
SELECT p FROM Producto p
JOIN p.categoria c
WHERE c.nombre = 'Electrónica'
```

- JOIN navega las relaciones ya definidas en las entidades (@ManyToOne, @OneToMany) — no necesitas escribir claves foráneas a mano.
- También existen LEFT JOIN (incluye registros sin coincidencia) y JOIN FETCH, que carga la relación de inmediato y evita el problema N+1 (múltiples consultas innecesarias).

TRANSFORMACIÓN

5 min

Funciones de agregación y ordenamiento

```
SELECT COUNT(p) FROM Producto p WHERE p.stock = 0

SELECT p FROM Producto p ORDER BY p.precio DESC

SELECT p.categoria, AVG(p.precio) FROM Producto p GROUP BY p.categoria
```

- COUNT, SUM, AVG, MAX, MIN funcionan igual que en SQL, aplicadas sobre atributos de la entidad.
- GROUP BY agrupa resultados por un atributo; ORDER BY define el orden (ASC / DESC).

TRANSFORMACIÓN

5 min

@Query en Spring Data JPA

Consultas personalizadas dentro de un JpaRepository

```
public interface ProductoRepository
        extends JpaRepository<Producto, Integer> {

    @Query("SELECT p FROM Producto p WHERE p.precio > :precioMin")
    List<Producto> buscarCostosos(@Param("precioMin") double precioMin);
}
```

- JpaRepository ya genera consultas automáticas a partir del nombre del método (ej. findByCategoria).
- @Query permite escribir JPQL a medida cuando la consulta es más compleja que un simple findBy...

TRANSFORMACIÓN

5 min

UPDATE y DELETE con JPQL

```
@Modifying
@Transactional
@Query("UPDATE Producto p SET p.stock = p.stock - :cantidad WHERE p.id = :id")
int descontarStock(@Param("id") Integer id, @Param("cantidad") int cantidad);
```

- @Modifying indica que la consulta cambia datos (UPDATE/DELETE) y no es un simple SELECT.
- El método retorna un int: el número de filas afectadas por la operación.
- **Toda operación de modificación SIEMPRE requiere una transacción activa — de lo contrario, Spring lanza una excepción.**

TRANSFORMACIÓN

6 min

¿Qué es una transacción? Propiedades ACID

Una transacción es un conjunto de operaciones que se ejecutan como una sola unidad indivisible: o se completan TODAS, o no se aplica NINGUNA.

A	C	I	D
Atomicidad	**Consistencia**	**Aislamiento**	**Durabilidad**
Todo o nada: si un paso falla, se revierte todo lo anterior.	La base de datos pasa de un estado válido a otro estado válido.	Transacciones simultáneas no se interfieren entre sí.	Una vez confirmada (commit), la operación persiste aunque el sistema falle después.

Ejemplo: una transferencia bancaria (debitar de A + acreditar en B) debe ser atómica — nunca a medias.

TRANSFORMACIÓN

6 min

@Transactional en Spring Boot

```
@Service
public class PedidoServiceImpl implements PedidoService {

    @Override
    @Transactional
    public void confirmarPedido(Pedido pedido) {
        productoRepository.descontarStock(
            pedido.getProductoId(), pedido.getCantidad());
        pedidoRepository.save(pedido);
        pagoService.registrarPago(pedido);
    }
}
```

- Spring abre la transacción antes de entrar al método y la confirma (commit) al salir con éxito.
- **Si ocurre una excepción no controlada (RuntimeException), Spring revierte (rollback) TODO automáticamente.**

TRANSFORMACIÓN

6 min

Buenas prácticas y atributos clave

Aplica @Transactional en la capa Service, NO en el Repository/DAO — es responsabilidad del Service definir los límites de la transacción.

`readOnly = true`	Optimiza el rendimiento en operaciones de solo lectura (consultas).
`rollbackFor = {...}`	Define qué tipos de excepción deben disparar el rollback de la transacción.
`propagation`	Define cómo se comporta la transacción cuando un método transaccional llama a otro (REQUIRED, REQUIRES_NEW...).
`isolation`	Controla qué tan “aisladas” están las transacciones concurrentes entre sí.

TRANSFORMACIÓN

5 min

Ejemplo práctico 1: catálogo con JOIN

TechStore Perú SAC necesita listar productos por categoría, ordenados por precio

```
public interface ProductoRepository
        extends JpaRepository<Producto, Integer> {

    @Query("SELECT p FROM Producto p JOIN p.categoria c " +
           "WHERE c.nombre = :nombreCategoria ORDER BY p.precio ASC")
    List<Producto> listarPorCategoriaOrdenado(
            @Param("nombreCategoria") String nombreCategoria);
}
```

Aplicación: la app móvil de TechStore muestra los productos más baratos primero al filtrar por categoría.

TRANSFORMACIÓN

5 min

Ejemplo práctico 2: compra con rollback

Si el pago falla, NADA debe quedar guardado

```
@Service
public class VentaServiceImpl implements VentaService {

    @Override
    @Transactional
    public void confirmarCompra(Long productoId, int cantidad) {
        productoRepository.descontarStock(productoId, cantidad);
        ventaRepository.save(new Venta(productoId, cantidad));
        pagoService.procesarPago(productoId, cantidad); // si falla...
    }                                          // ...todo se revierte
}
```

Si pagoService.procesarPago() lanza una excepción, Spring revierte el descuento de stock y el registro de venta — como si nunca hubiera pasado.

TRANSFORMACIÓN

5 min

Ejemplo práctico 3: reporte con agregación

El gerente de TechStore quiere el precio promedio por categoría

```
@Query("SELECT c.nombre, AVG(p.precio) FROM Producto p " +
       "JOIN p.categoria c GROUP BY c.nombre ORDER BY c.nombre")
List<Object[]> reportePrecioPromedioPorCategoria();
```

- Cada elemento del List<Object[]> resultante contiene [nombreCategoria, precioPromedio].
- Este tipo de consulta agregada evita traer TODOS los productos a memoria solo para calcular un promedio en Java.

PRÁCTICA

2 min

Ahora te toca a ti

Resuelve 3 retos progresivos usando JPQL y @Transactional

- Aplica lo aprendido hoy: JPQL, parámetros con nombre, @Query, @Modifying y @Transactional.
- Combínalo con lo visto en sesiones anteriores: entidades, EntityManager, JpaRepository.
- Trabaja sobre el proyecto TechStore Perú SAC (paquete pe.utp.techstore) que ya conoces.

INICIAL	INTERMEDIO	AVANZADO
Consulta JPQL con parámetro con nombre	Actualización masiva con @Query + @Modifying	Transacción completa con validación y rollback

PRÁCTICA

5 min

Ejercicio 1 · Nivel Inicial

CONSULTA JPQL CON PARÁMETRO CON NOMBRE

Escribe una consulta JPQL con parámetro con nombre en ProductoRepository que retorne todos los productos cuyo stock sea menor a un valor dado. Esta consulta servirá para generar una alerta de "productos por agotarse" en el panel del administrador de TechStore.

- Define el método en la interfaz ProductoRepository.
- Usa @Query con un parámetro con nombre (:stockMinimo).

PRÁCTICA

7 min

Ejercicio 2 · Nivel Intermedio

ACTUALIZACIÓN MASIVA CON @QUERY + @MODIFYING

Implementa un método en ProductoRepository, usando @Query y @Modifying, que aumente el precio de todos los productos de una categoría en un porcentaje dado. Recuerda anotar el método correspondiente en la capa Service con @Transactional.

- Usa la fórmula: nuevoPrecio = precio * (1 + porcentaje / 100).
- El método del repository debe retornar el número de filas afectadas (int).

PRÁCTICA

8 min

Ejercicio 3 · Nivel Avanzado

TRANSACCIÓN COMPLETA CON VALIDACIÓN Y ROLLBACK

Diseña un método transaccional registrarVenta() en VentaService que:

- Valide que haya stock suficiente antes de continuar.
- Descuente el stock con una consulta JPQL (@Modifying).
- Guarde el registro de la venta.
- **Lance una excepción si el stock es insuficiente, para forzar el rollback automático de TODO el proceso.**

PRÁCTICA

5 min

Solución: Ejercicios 1 y 2

Ejercicio 1 — Inicial

```
@Query("SELECT p FROM Producto p WHERE p.stock <
:stockMinimo")
List<Producto> buscarConStockBajo(
        @Param("stockMinimo") int stockMinimo);
```

El parámetro :stockMinimo evita hardcodear el umbral; se puede reutilizar con distintos valores.

Ejercicio 2 — Intermedio

```
@Modifying
@Query("UPDATE Producto p SET p.precio = " +
  "p.precio * (1 + :pct / 100.0) WHERE p.categoria.id = :catId")
int aumentarPrecioPorCategoria(
    @Param("catId") Integer catId, @Param("pct") double pct);
```

En el Service: @Transactional productoRepository.aumentarPrecioPorCategoria(id, 10.0);

PRÁCTICA

5 min

Solución: Ejercicio 3 (Avanzado)

```
@Service
public class VentaServiceImpl implements VentaService {

    @Override
    @Transactional
    public void registrarVenta(Long productoId, int cantidad) {
        Producto p = productoRepository.findById(productoId)
                .orElseThrow();
        if (p.getStock() < cantidad) {
            throw new IllegalStateException("Stock insuficiente");
        }
        productoRepository.descontarStock(productoId, cantidad);
        ventaRepository.save(new Venta(productoId, cantidad));
    }
}
```

Si el stock es insuficiente, la excepción interrumpe el método ANTES de tocar la base de datos: nada se guarda. Si el stock alcanza, ambas operaciones se confirman juntas.

CIERRE

4 min · Ronda relámpago

Cierre participativo: construyamos las conclusiones

"Ronda relámpago"

- En equipos de 3, resuman en UNA sola frase qué aprendieron hoy sobre JPQL.
- Y en otra frase, qué aprendieron sobre las transacciones (@Transactional).
- Un integrante por equipo comparte en voz alta — máximo 30 segundos.
- El docente anota las ideas clave en la pizarra para construir el resumen colectivo de la sesión.

CIERRE

5 min · Individual

Evaluación final: opción múltiple

Elige la alternativa correcta para cada pregunta

1. ¿Cuál es la diferencia principal entre JPQL y SQL?

a) JPQL solo funciona con MySQL b) JPQL trabaja con entidades y atributos, no con tablas y columnas c) JPQL no permite usar WHERE d) No hay diferencia

2. Si una RuntimeException se lanza dentro de un método @Transactional, ¿qué ocurre?

a) Spring ignora la excepción b) Se guarda parcialmente lo procesado c) Spring revierte (rollback) todos los cambios d) La aplicación se detiene

3. ¿Para qué sirve @Modifying junto con @Query?

a) Indica que la consulta modifica datos (UPDATE/DELETE) b) Mejora el rendimiento de SELECTs c) Crea una entidad automáticamente d) Deshabilita las transacciones

CIERRE

4 min · En pareja

Casos prácticos retadores

Caso 1: Un compañero programó descontarStock() con @Query y @Modifying, pero olvidó anotar el método del Service que lo invoca con @Transactional. Al probarlo en Postman, la aplicación lanza un error indicando que se requiere una transacción activa. Explica qué está pasando y cómo lo solucionarías.

Caso 2: En TechStore, dos clientes intentan comprar el último producto en stock casi al mismo tiempo. El sistema usa @Transactional en el método de venta. Explica, con tus palabras, por qué el uso de transacciones evita que ambos clientes "compren" el mismo producto agotado.



¡Excelente trabajo hoy!

Ya sabes construir consultas con JPQL y proteger las operaciones críticas de tus aplicaciones con @Transactional — una habilidad que usarás en cada proyecto profesional que involucre datos reales.

Gracias por tu participación activa en clase. Practica lo aprendido, resuelve la sesión de esta semana, y no olvides participar en el foro del curso — seguimos construyendo tu proyecto integrador la próxima sesión.